

PLAN

- **Programmation dynamique**

- Tas de Fibonacci
- Découpe de Barres
- Plus longue sous-séquence commune

- **Algorithme pour les graphes**

- Parcours en Largeur(BFS) PL
- Parcours en Profondeur(DFS)PP
- Tri Topologique TT
- Arbres couvrants minimaux(ACM)
- Dijkstra
- Réseau de flot

Algorithme

- Un algorithme prend en entrée des données et fournit un résultat permettant de donner la réponse à un problème.
- Algorithme $\longrightarrow$ Série d'opérations à effectuer.
 - Séquence
 - Parallèle
 - Réseau de processeur

Complexité

- Les algorithmes se différencient selon deux critères :
 - **Temps de calculs:** vitesse
 - **Ressources mémoire utilisées:** espace

Objectif:

- Avoir une idée de la difficulté des problèmes, †
- donner une idée du temps de calcul ou de l'espace mémoire nécessaire pour résoudre un problème

Complexité

- On peut dire qu'un algorithme est rapide s'il s'exécute en un temps polynomial: n , n^2 , n^4 .
 - Il existe plusieurs problèmes auxquelles on ne sais pas s'il existe de solutions rapides:
 - Le problème du sac à dos
 - Le problème du voyageur de commerce
- Il existe des solutions en $O(2^n)$. **1million de \$ pour la solution.**

Notation Asymptotique

- La notation **O** exprime la limite d'une fonction dans un facteur constant.

$$f = O(g(n)) \Leftrightarrow \exists n_0, \exists c \geq 0, f(n) \leq cg(n)$$

- La notation **O** reflète la courbe ou l'ordre croissant d'un algorithme.

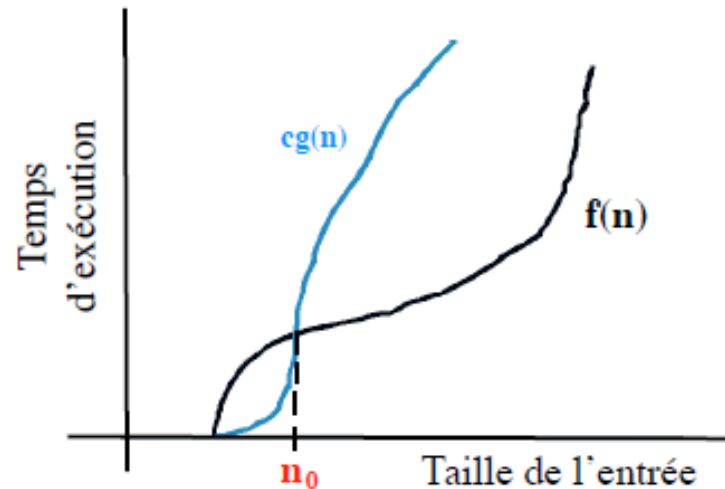


Fig1: Ordre croissant d'un algorithme

Notation Asymptotique

- On peut utiliser la notation O pour ordonner les fonctions à partir de leur taux de croissance.

$$1 \leq \log(n) \leq n \leq n \log(n) \leq n^2 \leq 2^n \leq n^n$$

La notation Ω

$$f = \Omega(g(n)) \Leftrightarrow \exists n_0, \exists c \geq 0, f(n) \geq cg(n)$$

Les notations Ω et O sont reflexives et transitives

Notation Asymptotique

- **La notation θ :** Étant donné des fonctions **$f(n)$ et $g(n)$** , on dit que $f(n)$ est $\theta(g(n))$ si $f(n)$ est $O(g(n))$ et $f(n)$ est $\Omega(g(n))$,
- *i.e. s'il existe des constantes $c_1 > 0$, $c_2 > 0$ et $n_0 \geq 1$ tel que*

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0$$

CHAPITRE II

TRI PAR INSERTION

Tri par Insertion

Tri-Insertion(A)

pour $j = 2$ à $A.\text{longueur}$

$\text{clé} = A[j]$

$i = j-1$

 tant que $(i > 0 \text{ et } A[i] > \text{clé})$

$A[i+1] = A[i]$

$i = i-1$

$A[i+1] = \text{clé}$

Tri par Insertion

Invariant de boucle et validité du tri par insertion:

Nous devons montrer trois choses concernant un invariant de boucle.

- **Initialisation:** Il est vrai avant la première itération de la boucle
- **Conservation:** S'il est vrai avant une itération de la boucle, il le reste avant l'itération suivante.
- **Terminaison:** Une fois terminée la boucle, l'invariant fournit une propriété utile qui nous aide à montrer la validité de l'algorithme.

Tri par Insertion

Comment ces propriétés s'appliquent au tri par insertion?

Initialisation :

- Quand $j=2$, le sous-tableau $A[1 \dots j-1]$ se compose de $A[1] \rightarrow$ trié

Conservation :

- le corps de la boucle **pour extérieure** fonctionne en déplaçant $A[j-1]$, $A[j-2]$, $A[j-3]$, etc. d'une position vers la droite jusqu'à ce qu'on trouve la bonne position pour $A[j]$.

Terminaison

- La boucle pour extérieure prend fin lorsque j dépasse $n \rightarrow j=n+1$
- En substituant $n+1$ à j dans la formulation de l'invariant de boucle, l'on a que le sous-tableau $A[1 \dots n]$ se compose des éléments qui appartenaient originellement à $A[1 \dots n]$ mais qui ont été triés depuis lors.

Analyse du Tri par Insertion

Tri-Insertion(A)	Coût	nombre de fois
pour j = 2 à A.longueur	c_1	n
clé = A[j]	c_2	n-1
i = j-1	c_3	n-1
tant que (i>0 et A[i]>clé)	c_4	$\sum_{j=2}^n t_j$
A[i+1] = A[i]	c_5	$\sum_{j=2}^n (t_j - 1)$
i = i-1	c_6	$\sum_{j=2}^n (t_j - 1)$
A[i+1] = clé	c_7	n-1

Analyse du Tri par Insertion

- Le temps d'exécution de l'algorithme est:

$$T(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7 (n-1)$$

- **Le cas le plus favorable:** $t_j = 1$ pour $j = 2, 3, \dots, n$,

$$T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7) \rightarrow an + b \rightarrow O(n)$$

Analyse du Tri par Insertion

- $T(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7 (n-1)$

Le cas le plus défavorable: $t_j = j$ pour $j = 2, 3, \dots, n$,

Avec $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$ et $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$

$$T(n) = (c_4 + c_5 + c_6)n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7)n$$

$$\rightarrow an^2 + bn + c \rightarrow O(n^2)$$

Analyse du Tri par Insertion

- $T(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7 (n-1)$

Le cas le plus défavorable: $t_j = j$ pour $j = 2, 3, \dots, n$,

Avec $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$ et $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$

$$T(n) = (c_4 + c_5 + c_6)n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7)n$$

$$\rightarrow an^2 + bn + c \rightarrow O(n^2)$$

CHAPITRE III

METHODE DIVISER POUR REGNER

Méthode Diviser pour régner

Diviser pour régner :

- **Diviser** : le problème en un certain nombre de sous-problèmes.
- **Régner** : sur les sous-problèmes en les résolvant de manière récursive. Si la taille d'un sous-problème est suffisamment réduite, on peut toutefois le résoudre directement.
- **Combiner** : les solutions des sous-problèmes pour produire la solution du problème originel.

Méthode Diviser pour régner

Tri par fusion

- **Diviser** : Diviser la suite de n éléments à trier en deux sous-suites de $n/2$ éléments chacune.
- **Régner** : Trier les deux sous-suites de manière récursive en utilisant le tri par fusion.
- **Combiner** : Fusionner les deux sous-suites triées pour produire la réponse triée.

Méthode Diviser pour régner

```
FUSION(A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  //création des deux tableaux L[1 .. n1 + 1] et R[1 .. n2 + 1]
  pour i = 1 à n1
    L[i] = A[p + i - 1]
  pour j = 1 à n2
    R[j] = A[q + j]
  L[n1 + 1] = ∞
  R[n2 + 1] = ∞
  i = 1
  j = 1
  pour k = p à r
    si L[i] ≤ R[j]
      A[k] = L[i]
      i = i + 1
    sinon A[k] = R[j]
      j = j + 1
    k = k + 1
```

Méthode Diviser pour régner(Tri-Fusion)

TRI-FUSION(A,p,r)

Si $p < r$

$$q = \left\lfloor \frac{(p+r)}{2} \right\rfloor$$

TRI-FUSION (A,p,q)

TRI-FUSION (A,q+1,r)

TRI-FUSION (A, p, q, r)

Analyse de l'algorithme diviser pour régner

- soit $T(n)$ le temps d'exécution d'un problème de taille n .
Si $n \leq c$ pour une certaine constante $c \rightarrow O(1)$.
- Si on divise le problème en **a** sous problèmes, la taille de chacun étant **1/b** de la taille du problème initial,
- Si on prend **D(n)** le temps pour diviser le problème en sous problèmes et un temps **C(n)** pour construire la solution finale, on obtient:

$$T(n) = \begin{cases} \theta(1) & \text{si } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{sin on} \end{cases}$$

Analyse du Tri-Fusion

Diviser: calculer le milieu du sous-tableau, ce qui consomme un temps constant. Donc $D(n) = \theta(1)$.

Régner:

- On résout récursivement deux sous-problèmes, chacun ayant la taille $n/2$, ce qui contribue pour $2T(n/2)$ au temps d'exécution

Combiner:

- FUSION sur un sous-tableau à n éléments prenait un temps $\theta(n) = C(n)$.
- On ajoute le terme $2T(n/2)$ aux fonctions $D(n)$ et $C(n)$, on obtient:

$$T(n) = \begin{cases} O(1) & \text{si } n = 1 \\ 2T(n/2) + \theta(n) & \text{si } n > 1 \end{cases}$$

Méthode Générale

- La méthode générale donne des solutions pour résoudre des récurrences de la forme:

$$T(n) = aT(n/b) + f(n) \text{ avec } a \geq 1 \text{ et } b > 1$$

- $f(n) = D(n) + C(n)$
- Dans le cas du tri-Fusion: $a=2$, $b=2$ et $f(n) = \theta(n)$

Théorème Générale

- La méthode générale donne des solutions pour résoudre des récurrences de la forme:

Soit $a \geq 1$ et $b > 1$ et $f(n)$ une fonction,

$$T(n) = aT(n/b) + f(n),$$

- $T(n)$ peut être bornée de la façon suivante:

$f(n)$	$T(n)$
$O(n^{\log_b a - \varepsilon})$	$\theta(n^{\log_b a})$
$\theta(n^{\log_b a})$	$\theta(n^{\log_b a} \cdot \log(n))$
$\Omega(n^{\log_b a + \varepsilon})$	$\theta(f(n))$